

Controle de Despesas - Webapp

Aplicação web desenvolvida em Flask para gerenciamento de despesas pessoais com exportação para Google Sheets.

Tecnologias Utilizadas

- **Backend:** Python 3.11, Flask, SQLAlchemy
- **Banco de Dados:** SQLite
- **Frontend:** Tailwind CSS (via CDN), Font Awesome
- **Integração:** Google Sheets API (gsread, oauth2client)

Funcionalidades

- ☒ Cadastro de despesas com descrição, valor, categoria, data e observações
- ☒ Listagem de todas as despesas cadastradas
- ☒ Edição de despesas existentes
- ☒ Exclusão de despesas
- ☒ Cálculo automático do total de despesas
- ☒ Exportação de dados para Google Sheets
- ☒ Interface responsiva e moderna com Tailwind CSS
- ☒ Categorias predefinidas (Alimentação, Transporte, Moradia, etc.)

Estrutura do Projeto

Plain Text

```
despesas-app/  
├── app.py           # Arquivo principal da aplicação  
├── models.py        # Modelos de dados (SQLAlchemy)  
├── config.py        # Configurações da aplicação  
├── templates/  
│   ├── base.html   # Template base  
│   ├── index.html  # Página principal  
│   ├── adicionar.html # Formulário de adição  
│   └── editar.html  # Formulário de edição  
├── static/  
│   ├── css/  
│   └── js/          # Arquivos estáticos
```

```
|— venv/                                # Ambiente virtual Python
|— despesas.db                          # Banco de dados SQLite (gerado
automaticamente)
|— credentials.json                     # Credenciais do Google (não incluído)
|— credentials.json.example             # Exemplo de credenciais
|— .gitignore                           # Arquivos ignorados pelo Git
|— README.md                            # Este arquivo
```

Instalação e Configuração

1. Pré-requisitos

- Python 3.11 ou superior
- pip (gerenciador de pacotes Python)

2. Instalação

Bash

```
# Clone ou baixe o projeto
cd despesas-app

# Crie um ambiente virtual
python3.11 -m venv venv

# Ative o ambiente virtual
source venv/bin/activate # Linux/Mac
# ou
venv\Scripts\activate   # Windows

# Instale as dependências
pip install flask flask-sqlalchemy gspread oauth2client
```

3. Executar a Aplicação

Bash

```
# Ative o ambiente virtual (se ainda não estiver ativo)
source venv/bin/activate

# Execute a aplicação
python app.py
```

A aplicação estará disponível em: `http://localhost:5000`

Configuração do Google Sheets

Para usar a funcionalidade de exportação para Google Sheets, siga os passos abaixo:

1. Criar um Projeto no Google Cloud Console

1. Acesse [Google Cloud Console](#)
2. Crie um novo projeto ou selecione um existente
3. Ative as APIs necessárias:
 - Google Sheets API
 - Google Drive API

2. Criar Credenciais de Conta de Serviço

1. Vá em **APIs & Services > Credentials**
2. Clique em **Create Credentials > Service Account**
3. Preencha os dados da conta de serviço
4. Após criar, clique na conta de serviço criada
5. Vá em **Keys > Add Key > Create New Key**
6. Escolha o formato **JSON** e baixe o arquivo
7. Renomeie o arquivo baixado para `credentials.json`
8. Coloque o arquivo `credentials.json` na raiz do projeto

3. Compartilhar a Planilha

1. Abra o arquivo `credentials.json` e copie o valor do campo `client_email`
2. No Google Sheets, crie ou abra a planilha que deseja usar
3. Clique em **Compartilhar** e adicione o email copiado com permissão de **Editor**

4. Usar a Exportação

1. Na aplicação, clique no botão **Exportar para Google Sheets**
2. Digite o nome da planilha (deve ser o mesmo nome da planilha no Google Sheets)
3. Clique em **Exportar**
4. Os dados serão enviados para a planilha

Categorias de Despesas

As seguintes categorias estão predefinidas na aplicação:

- Alimentação
- Transporte
- Moradia
- Saúde
- Educação
- Lazer
- Vestuário
- Contas
- Outros

Você pode personalizar essas categorias editando o arquivo `config.py`.

API REST

A aplicação também fornece uma API REST simples:

Listar todas as despesas (JSON)

Plain Text

GET `/api/despesas`

Retorna todas as despesas em formato JSON.

Personalização

Alterar Categorias

Edite o arquivo `config.py` e modifique a lista `CATEGORIAS`:

Python

```
CATEGORIAS = [  
    'Sua Categoria 1',  
    'Sua Categoria 2',
```

```
# ...  
]
```

Alterar Porta do Servidor

Edite o arquivo `app.py` na última linha:

Python

```
app.run(debug=True, host='0.0.0.0', port=5000) # Altere 5000 para a porta  
desejada
```

Alterar Tema/Cores

Os templates usam Tailwind CSS. Você pode personalizar as cores editando as classes CSS nos arquivos HTML em `templates/`.

Segurança

⚠ Importante:

- Nunca compartilhe o arquivo `credentials.json` publicamente
- O arquivo já está incluído no `.gitignore` para evitar commits acidentais
- Em produção, use variáveis de ambiente para armazenar credenciais sensíveis
- Desative o modo debug (`debug=False`) em produção

Desenvolvimento

Estrutura do Banco de Dados

A tabela `despesas` possui os seguintes campos:

- `id` (Integer, Primary Key): Identificador único
- `descricao` (String): Descrição da despesa
- `valor` (Float): Valor da despesa
- `categoria` (String): Categoria da despesa
- `data` (Date): Data da despesa
- `observacoes` (Text): Observações adicionais (opcional)
- `criado_em` (DateTime): Data/hora de criação do registro

Adicionar Novas Funcionalidades

Para adicionar novas rotas, edite o arquivo `app.py` :

Python

```
@app.route('/sua-rota')
def sua_funcao():
    # Sua lógica aqui
    return render_template('seu_template.html')
```

Solução de Problemas

Erro: "Port 5000 is in use"

Outro processo está usando a porta 5000. Você pode:

1. Parar o processo que está usando a porta:

Bash

```
fuser -k 5000/tcp
```

2. Ou alterar a porta da aplicação no arquivo `app.py`

Erro ao exportar para Google Sheets

- Verifique se o arquivo `credentials.json` está na raiz do projeto
- Verifique se a planilha foi compartilhada com o email da conta de serviço
- Verifique se as APIs do Google Sheets e Drive estão ativadas no projeto

Banco de dados não é criado

O banco de dados é criado automaticamente na primeira execução. Se houver problemas:

Bash

```
python -c "from app import app, db; app.app_context().push();
db.create_all()"
```

Licença

Este projeto é de código aberto e está disponível para uso pessoal e educacional.

Suporte

Para dúvidas ou problemas, abra uma issue no repositório do projeto.

Desenvolvido com ❤️ usando Flask e Tailwind CSS